

Design notes

The core decision: JSONB vs. typed columns

Every incoming field gets one of two homes. The test is **how we will use it**, not what it is:

Promote to a typed column if we expect to *filter, sort, join, aggregate, or expose it directly* to the CMS / analysts. **Leave it in JSONB** if it's bulk payload we want to retain but query rarely, ad hoc, or not at all yet.

This project does **both at once** rather than choosing globally:

- `raw jsonb NOT NULL` holds the **complete** API response for every row. We never discard data, so a field we ignore today is still there tomorrow.
- A curated set of high-value fields is **promoted** to typed, indexed columns (`total_population` , `median_household_income` , ...). The promotion map is one list in `variables.py` .

Why this split, concretely

Force	Implication
ACS has <i>tens of thousands</i> of variables per dataset	Promoting all of them is absurd; a single JSONB column absorbs the long tail.
We filter/sort on a <i>small, stable</i> subset	Those belong in typed columns with btree indexes — JSONB extraction + casting per row is slow and unindexed by default.
Requirements change	Promotion is a cheap, additive migration (add a column, backfill from <code>raw</code>); we never re-ingest because the source data is already in JSONB.
Downstream (Payload, modeling) wants clean types	Typed columns give real <code>bigint / integer</code> , not text-in-JSON.

Promotion is reversible and incremental

Because `raw` always holds everything, "promote field X" is just:

```
ALTER TABLE acs_observations ADD COLUMN housing_units bigint;  
UPDATE acs_observations SET housing_units = (raw ->> 'B25001_001E')::bigint;  
CREATE INDEX ON acs_observations (housing_units);
```

No new API calls. This is the main payoff of keeping the raw payload.

Indexing strategy

- **btree** on promoted columns that drive filters/sorts (`median_household_income`, `total_population`), as *partial* indexes (`WHERE col IS NOT NULL`) since ACS has many null/suppressed values — smaller index, no dead entries.
- **GIN with `jsonb_path_ops`** on `raw`. This serves containment (`@>`) queries against un-promoted fields (e.g. `raw @> '{"state":"06"}'`). `jsonb_path_ops` is chosen over the default `jsonb_ops` because we only need containment, and it produces a smaller, faster index. The trade-off: it doesn't support key-exists (`?>`) operators — acceptable here.
- **Composite** (`geography`, `year`) to scope queries to a level/vintage quickly.

Idempotency & incremental loads

- **Natural key:** (`dataset`, `year`, `geography`, `geoid`). The GEOID is built by concatenating Census geo codes (state+county+tract...), mirroring the official convention. The loader upserts with `ON CONFLICT ... DO UPDATE`, so re-running a load is safe and self-healing — no duplicates, latest values win.
- **Watermark:** `load_runs` records every run's params, status, row count, and errors. The CLI skips a (`dataset`, `year`, `geography`) that already has a `success` row unless `--force`, so a scheduled daily job is nearly free and only genuinely new vintages do work.
- **Bounded transactions:** the loader commits per 500-row batch, so a failure midway leaves committed batches in place (safe — they'll just upsert again) and records a `failed` run for observability.

Scaling the API pulls

The Census API has **no cursor/offset pagination**. Large geographies are handled by **fanning out over parent geography**: you can't fetch every tract in the country at once, so the client issues one request per state (~50 calls), each throttled (min-interval limiter) and retried with

exponential backoff + jitter on `429 / 5xx` /network errors. Results stream back through a single generator the loader consumes lazily, so memory stays flat regardless of total row count.

Census also caps a request at 50 variables; the curated set stays under that. For wide pulls the same `client.fetch` chunking pattern extends to batching variable codes into ≤ 50 -column requests and merging on GEOID.

Why no ORM

The whole point of the project is the storage model and indexing. Plain SQL + psycopg keeps the JSONB adaptation, upsert, and DDL fully visible and makes the batched-upsert hot path easy to reason about.

Applying the pattern to the other sources

- **Voter files** — bulk flat files, not an API. Same JSONB+typed split: promote the fields you match/segment on (county, precinct, registration status, vote-history flags), keep the rest in JSONB. Natural key = voter ID + file vintage; the same `load_runs` watermark drives incremental refreshes.
- **CCES / survey & crosswalk** — wide respondent tables. Promote weights, key demographics, and crosswalk keys (FIPS/PUMA) for joins to ACS; the long tail of survey items lives in JSONB until a model needs a specific item.